WORKING WITH OPENAI & ANTHROPIC REASONING MODELS

O'Reilly Live Training — Quick Reference Cheatsheet

Lucas Soares I GPT-5.6 · Claude Extended Thinking · DeepSeek R1 I 2026

1. WHAT IS A REASONING / THINKING MODEL?

Core Characteristics

- **Chain-of-Thought (CoT)** — spends time "thinking" before answering
- **Intermediate steps** — shows work, transparent & verifiable
- **Variable compute** — allocates more tokens for harder problems
- **RL-trained** — rewarded for step-by-step logical reasoning
- **Self-critique** — revises and verifies its own answers

System 1 vs System 2 Thinking

Traditional LLMs	Reasoning Models
Fast, intuitive (System 1)	Slow, methodical (System 2)
Pattern matching	Step-by-step analysis
Fixed forward passes	Variable compute time
General RLHF	RL with reasoning rewards

OpenAI gpt-5.6 family

- gpt-5.6-sol / -terra / -luna (intelligence → cost/latency)
- Proprietary, API-only
- Reasoning effort: `none` / `low` / `medium` / `high` / `xhigh` / `max`
- Responses API (stateful)
- Vision support (native)

Anthropic Claude

- Claude Opus 5 / Sonnet 5
- Adaptive thinking (on by default) + `output_config.effort`
- Effort: `low` / `medium` / `high` / `xhigh` / `max`
- Haiku 4.5: classic `thinking.budget_tokens` only
- Summarized thinking via `display: "summarized"`

DeepSeek R1

- 671B params, MIT license
- Open-source, self-hostable
- Distilled: 1.5B–70B params
- Strong math reasoning
- Hugging Face ecosystem

2. WHEN TO USE REASONING MODELS

Use Reasoning Models When…

- Accuracy & reliability > speed
- Complex, ambiguous, multi-step problems
- Explainable reasoning chains required
- Agentic planning & orchestration ("the planner")
- Finding relationships across large datasets
- Code review & architecture decisions
- LLM-as-judge evaluation tasks
- Navigating dense unstructured documents
- Visual reasoning over complex charts

Use Traditional LLMs When…

- Speed and cost are primary concerns
- Well-defined, straightforward tasks
- Task *execution* (not planning)
- Creative writing & conversational chat
- Content generation at volume
- Low-latency interactive applications
- Routing, summarization, triage
- gpt-5.6-sol planner → gpt-5.6-luna "doer" steps

3. REAL-WORLD USE CASES (WITH RESULTS)

Proven Enterprise Results

- **Tax research** — 4× performance boost (multi-doc reasoning, Blue J)
- **Code review** — 3× conversion increase (CodeRabbit)
- **LLM-as-judge** — F1 score 0.12 → 0.74 (Braintrust)
- **Image classification** — 88% vs 50% accuracy (SafetyKit)
- **Legal analysis** — 52% stronger on complex contracts (Hebbia)

Application Domains

- **Document Analysis** — contracts, financial statements, legal
- **Code** — debugging, architecture, one-shotting entire files
- **Decision Support** — financial analysis, risk assessment
- **Agentic Workflows** — gpt-5.6-sol plans, gpt-5.6-luna executes
- **Research** — hypothesis testing, paper synthesis
- **Visual Reasoning** — architectural drawings, dense charts

4. CHOOSING THE RIGHT MODEL — KEY CRITERIA

Selection Criteria & Trade-offs

Criterion	What to Evaluate	Trade-off
Reasoning Quality	CoT capabilities, logical consistency, benchmarks (MATH, GSM8K, MMLU)	Higher quality = higher cost & latency
Embedded Knowledge	Domain expertise, factual accuracy, knowledge cutoff date	Freshness vs. depth of specialization
Context Window	Document length, multi-turn conversation capacity	Larger window = more compute cost
Deployment	API-only vs. self-hostable; cloud vs. on-premises	Control & privacy vs. maintenance overhead
Cost	Cost per token, inference speed, hardware requirements	Performance vs. operational cost
Specialization	Code, math, multimodal, multilingual support	Generality vs. best-in-class for specific task

5. HOW TO PROMPT REASONING MODELS

OpenAI gpt-5.6 (sol / terra / luna)

- **Write briefs, not prompts** — 10× more context than you think
- **Focus on WHAT, not HOW** — let the model plan its approach
- **Keep it direct** — no "think step by step" needed (redundant)
- **Do NOT chain-of-thought prompt** — model reasons internally
- **Use delimiters** — markdown, XML tags, section titles
- **Zero-shot first**, then few-shot if needed
- **Define success criteria** — let model self-evaluate
- **Developer messages** (not system messages)
- **Reasoning effort API**: `none` / `low` / `medium` / `high` / `xhigh` / `max`
- **Responses API** with `store=true` for multi-turn + tool calls

Anthropic Claude (Opus 5 / Sonnet 5)

- **Effort** — `output_config.effort`: low → xhigh → max, default high
- **Adaptive thinking** — on by default; no config needed to reason
- **Start minimal** — don't say "think step by step" (redundant)
- **Streaming required** for large outputs (improves UX)
- **Tool use**: set `tool_choice: auto`
- **Summarized thinking** — opt in with `display: "summarized"` (defaults to omitted)
- **Show progress** — stream thinking blocks to reduce perceived latency
- **Haiku 4.5** — no effort/adaptive support; use classic `budget_tokens`

Key Mental Model: gpt-5.6 is a "Report Generator", not a Chat Model

Treat it like email, not texting. Give full context upfront (company background, schemas, code, constraints). Ask for one specific output. Define what "good" looks like. It will one-shot entire files — but won't iteratively chat its way to a solution.

6. API QUICK REFERENCE

OpenAI — Responses API (gpt-5.6)

```
from openai import OpenAI
client = OpenAI()

response = client.responses.create(
    model="gpt-5.6",
    reasoning={"effort": "high"},
    input=[{
        "role": "user",
        "content": "Your detailed brief here..."
    }],
    store=True  # enable multi-turn reasoning
)
print(response.output_text)
```

Anthropic — Effort-based Reasoning (Claude Opus 5)

```
import anthropic
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-5",
    max_tokens=16000,
    output_config={"effort": "high"},
    messages=[{
        "role": "user",
        "content": "Your problem here..."
    }]
)
# Thinking is on by default (adaptive); opt in to seeing it with
# thinking={"type": "adaptive", "display": "summarized"}
print(response.content[-1].text)
```

OpenAI — Streaming Reasoning

```
with client.responses.stream(
    model="gpt-5.6",
    reasoning={"effort": "medium"},
    input=[{"role": "user", "content": brief}]
) as stream:
    for event in stream:
        # Handle text deltas in real-time
        if event.type == "response.text.delta":
            print(event.delta, end="", flush=True)
```

Anthropic — Streaming Extended Thinking (Claude Haiku 4.5)

```
with client.messages.stream(
    model="claude-haiku-4-5-20251001",
    max_tokens=16000,
    thinking={"type":"enabled","budget_tokens":8000},
    messages=[{"role":"user","content":problem}]
) as stream:
    for event in stream:
        if event.type == "content_block_delta":
            # thinking_delta or text_delta
            print(event.delta)
```

`budget_tokens` is Haiku 4.5-only — Opus 5 / Sonnet 5 use adaptive thinking + `output_config.effort` instead.

7. DEEPSEEK R1 VS OPENAI GPT-5.6 — COMPARISON

Model Comparison

Dimension	DeepSeek R1	OpenAI gpt-5.6
License	Open-source (MIT)	Proprietary
Size	671B parameters (full)	Undisclosed
Access	API + self-host + Hugging Face	OpenAI API only
Cost	Free (self-host) / API pricing	Pay-per-token
Math	Particularly strong	Excellent (occasional gaps)
Customization	Full: quantize, fine-tune, prune	Limited
Deployment	Cloud + on-premises + edge	Cloud only
Ecosystem	Growing OSS community	Mature commercial support

Distilled Models (DeepSeek-R1-Distill)

Available in 1.5B → 70B parameter sizes. Achieve 80–95% of teacher model performance. **Llama-70B distill:** 94.5% on MATH-500, 86.7% on AIME 2024. **Qwen-32B distill** outperforms OpenAI-o1-mini across multiple benchmarks. Use when: resource-constrained, edge deployment, latency-sensitive, or cost-sensitive applications.

8. LIMITATIONS & MITIGATION STRATEGIES

Known Limitations

- **Latency** — significantly longer response times
- **Cost** — more reasoning tokens = higher cost
- **Domain brittleness** — inconsistent across subject areas
- **Over-confidence** — may present flawed reasoning confidently
- **Knowledge cutoff** — bounded by training data
- **Not for** — creative writing in specific voices
- **Not for** — building entire apps in one shot
- **Context limits** — struggles with very long reasoning chains

Mitigation Strategies

- **Hybrid approach** — reasoning model plans, GPT executes
- **Fallback mechanisms** — alternate path when reasoning fails
- **Human-in-the-loop** — critical decisions need human review
- **Progressive UI** — stream thinking steps to reduce perceived latency
- **Model ensembles** — consensus from multiple models
- **Domain testing** — benchmark on your specific task type
- **Distilled models** — for cost/latency-sensitive deployments

9. COURSE NOTEBOOKS

01_build_a_reasoning_model_r1_from_scratch.ipynb

- Full R1 pipeline in the paper's sequence
- Foundations → base → R1-Zero → cold-start SFT → GRPO
- Rejection-sampling SFT → distillation
- One metric (pass-rate) climbs the whole way
- Practical half → `02_reproducibility_cheap_vs_flagship.ipynb`

provider_openai_thinking_parameters.ipynb

- GPT-5.6 Responses API deep-dive
- Reasoning effort levels (none→max)
- Verbosity control (independent)
- Tool use + web search integration
- Structured outputs with Pydantic

provider_anthropic_extended_thinki

- Claude Opus 5 / Sonnet 5 adaptive thinkin
- `output_config.effort` + classic `budget_tokens` (Haiku 4.5)
- Streaming implementation
- Summarized thinking display
- Tool use with extended thinking

10. QUICK DECISION FRAMEWORK

Model Selection Tree

```
Need reasoning model?
├─ Speed > accuracy? → gpt-5.6-luna (effort=none) / traditional LLM
├─ Complex + ambiguous problem?
│   ├─ Need on-premises? → DeepSeek R1 (self-host)
│   ├─ Cost-sensitive? → gpt-5.6-luna / Distilled model (32B)
│   ├─ Best accuracy? → gpt-5.6-sol (xhigh/max) or Claude Opus 5
│   └─ Vision needed? → gpt-5.6-sol (native vision)
└─ Agentic workflow?
    ├─ Planner role → gpt-5.6 / Claude Opus 5 (extended thinking)
    └─ Executor role → gpt-5.6-luna / Claude Sonnet 5 (fast)
```

Reasoning Effort Guidelines

Effort	Use When
`low`	Well-defined, structured tasks
`medium`	Most standard complex problems
`high`	Multi-doc analysis, hard math
`xhigh`	Near-max accuracy, complex multi-step
`max`	Maximum reasoning, most complex tasks

Claude Opus 5 / Sonnet 5: use the effort table above (`output_config.effort`) — thinking is adaptive and on by default.
Claude Haiku 4.5 budget tokens (no effort support): 1K–2K = quick · 8K–16K = standard · 32K+ = deep analysis

KEY TAKEAWAYS

- **Reasoning models** trade speed for accuracy via internal CoT
- **Write briefs** for gpt-5.6 — 10× context, clear goals, one output
- **Effort / budget tokens** control reasoning depth (start minimal)
- **Don't say** "think step by step" — redundant for both APIs
- **DeepSeek R1** is open-source, self-hostable, MIT licensed
- **Distilled models** reach 80–95% of teacher model performance
- **Planner + doer pattern** — gpt-5.6-sol plans, gpt-5.6-luna executes
- **Stream responses** to manage latency expectations in production
- **Hybrid fallbacks** — design for when reasoning fails or times out
- **Benchmark on YOUR domain** — performance varies significantly by task type